Topics

🧱 1. Python Foundations (Beginner)

Goal: Get comfortable writing Python programs

📌 Topics

- **Variables, Data Types, Operators** — Store and manipulate values
- **Control Structures** — Make decisions (`if/else`) and loops (`for`, `while`)
- **Data Structures** — Built-in collections: `list`, `tuple`, `dict`, `set`
- **Functions** — Reusable blocks of code with parameters and return values
- **Error Handling** — Catch runtime errors using `try/except`
- **File Handling** — Read and write text files with `open()`

📚 Resources

- 📹 Python Full Course for Beginners (freeCodeCamp) — 4 hours, covers fundamentals
- 📘 Python Docs – Tutorial
- 📝 W3Schools Python Basics

🧠 2. Core Python (Intermediate)

Goal: Write modular and structured Python code

📌 Topics

- **OOP (Classes, Inheritance, Polymorphism)**
- **Modules & Packages** (`import`, `__init__.py`)

- **Custom Exceptions** (`class MyError(Exception):`)
- **Iterators & Generators** (`__iter__`, `yield`)
- **Comprehensions** (List/Dict/Set one-liners)
- **Decorators** (`@mydecorator`) and **Context Managers** (`with`)
- **Type hints** (`def func(x: int) -> str:`)
- **Testing** (`unittest`, `pytest`)
- **Virtual Environments** (`python -m venv venv`, `pip install`)

📚 Resources

- 📹 Intermediate Python Programming Course (freeCodeCamp)
- 📘 Python OOP Documentation
- 📘 Python Packaging Guide
- 📘 pytest Docs

🌐 3. Web Development with Python

Goal: Build real websites and REST APIs

🌍 Web Basics

- Understand how the web works: HTTP requests, responses, headers, status codes, HTTPS
- HTML, CSS, JavaScript — to build the frontend of your web apps

📚 Resources

- 📹 Web Development Fundamentals (freeCodeCamp)
- 📘 MDN Web Docs — Web Basics

⚙️ Python Web Frameworks

🧩 Flask

- Lightweight, great for beginners & small projects
- Routing, templates (Jinja2), forms, sessions

📚

- 📹 Flask Full Course (freeCodeCamp)
- 📘 Flask Docs

🧩 Django

- Batteries included — admin panel, ORM, auth system
- MVT architecture, scalable for big projects

📚

- 📹 Django Crash Course (Traversy Media)
- 📘 Django Docs

🧩 FastAPI

- Modern, async-friendly, perfect for APIs
- Built-in OpenAPI docs

📚

- 📹 FastAPI Course (freeCodeCamp)
- 📘 FastAPI Docs

🗃️ Web-Dev Essentials

- **Databases**: SQLite/PostgreSQL + ORM (SQLAlchemy, Django ORM)
- **Authentication & Authorization** (sessions, JWTs)
- **REST APIs** (CRUD endpoints, HTTP methods)
- **Async Programming** (`async/await`)
- **Deployment** (Heroku, Render, Railway)

📚

- 📘 SQLAlchemy Docs
- 📹 PostgreSQL + Python (Programming with Mosh)
- 📘 Deploy Django App on Render

💻 4. Software/Desktop Development

Goal: Build GUI-based apps

🖥️ Tkinter (Built-in)

- Simple, good for small tools
- Buttons, labels, text fields, menus

📚

- 📹 Tkinter Full Course (freeCodeCamp)
- 📘 Tkinter Docs

🖥️ PyQt / PySide

- Professional looking apps, powerful
- Complex layouts, dialogs, events

📚

- 📹 PyQt5 Tutorial (Codemy)
- 📘 PyQt Docs

🖥️ Kivy

- Cross-platform, touch-friendly (Android, iOS, desktop)

📚

- 📹 Kivy Crash Course
- 📘 Kivy Docs

🗃️ Packaging your App

- Convert .py to .exe / .app
- Tools: **PyInstaller**, **cx_Freeze**

📚

- 📘 PyInstaller Docs
- 📹 How to Make Python EXE (Tech With Tim)

⚙️ 5. Supporting Technologies

Goal: Make your apps production-ready

- **Databases**: SQL (PostgreSQL, MySQL, SQLite), NoSQL (MongoDB, Redis)
- **Git + GitHub** (version control)
- **APIs**: Using `requests` library
- **Environment Config**: `.env`, `dotenv`, `configparser`
- **Logging & Debugging**: `logging`, `pdb`

📚

- 📹 Git & GitHub Crash Course (Traversy Media)
- 📘 Requests Docs
- 📘 Python Logging Docs

⚡ 6. Advanced Concepts & Tools

Goal: Handle big projects & improve performance

- **Asynchronous Programming**: `asyncio`, `aiohttp`, `multiprocessing`
- **Design Patterns**: MVC, Singleton, Factory, Observer
- **Security**: SQL Injection, XSS/CSRF prevention, password hashing

- **Testing + CI/CD**: `pytest`, GitHub Actions, Docker. UNIT Testing
- **Containerization & Deployment**: Docker + Cloud platforms
- **Performance**: `cProfile`, memory optimization
- **Documentation**: `Sphinx`, docstrings

📚

- 📹 Async Python Course (freeCodeCamp)
- 📘 Design Patterns in Python
- 📹 Docker + Python (freeCodeCamp)
- 📘 Sphinx Docs

🧩 7. Commonly Used Python Libraries

Purpose	Library
HTTP Requests	`requests`, `httpx`
Web Scraping	`BeautifulSoup`, `Selenium`
Databases	`SQLAlchemy`, `psycopg2`, `pymongo`
GUI	`Tkinter`, `PyQt`, `Kivy`
Testing	`pytest`, `unittest`
Scheduling	`schedule`, `apscheduler`
Files	`openpyxl` (Excel), `pdfplumber` (PDF), `csv`, `json`

📍 Project Ladder (for practice)

Stage	Projects
Beginner	CLI To-Do app, File Organizer
Intermediate	Flask blog, Tkinter contact book
Advanced	FastAPI SaaS backend, PyQt Inventory System

Roadmap

🗺️ 8-Week Python Developer Roadmap (Web + Software)

📅 Week 1: Python Foundations

Goal: Get comfortable writing Python scripts and understand core syntax.

✅ Topics

- Python setup & IDE (VS Code / PyCharm)
- Variables, Data Types, Operators
- Input & Output
- Control structures: `if/else`, `for`, `while`
- Lists, Tuples, Sets, Dictionaries
- Functions & Scope
- Basic file handling (`open`, `read`, `write`)

🎯 Practice

- Write a simple calculator CLI app
- Write a program to manage a list of tasks (To-Do CLI)

📚 Resources

- 📹 Python for Beginners – freeCodeCamp
- 📘 Python Official Tutorial

📅 Week 2: Core Python

Goal: Learn to write modular, reusable, and maintainable code.

✅ Topics

- Object-Oriented Programming (OOP)
    - Classes, Objects, Methods, Inheritance, Polymorphism
- Modules & Packages (`import`, `__init__.py`)
- Exception Handling (custom exceptions)
- Iterators & Generators (`yield`)
- List/Dict Comprehensions
- Decorators & Context Managers (`@decorator`, `with`)

🎯 Practice

- Build a **Contact Book** CLI (OOP-based)
- Create your own module with utility functions
- Implement a custom exception in a program

📚 Resources

- 📹 Intermediate Python – freeCodeCamp
- 📘 Python Classes Documentation

📅 Week 3: Python Ecosystem & Testing

Goal: Learn to work in professional environments.

✅ Topics

- Virtual Environments (`venv`, `pip install`, `requirements.txt`)
- Unit Testing (`unittest` or `pytest`)
- Logging (`logging` module)
- Reading/Writing JSON, CSV, Excel files
- Introduction to APIs (using `requests`)

🎯 Practice

- Write tests for your contact book app
- Fetch weather data from a public API and display it
- Create a script to read/write CSV and Excel files

📚 Resources

- 📘 Pytest Docs
- 📘 Requests Docs
- 📹 Working with APIs in Python

📅 Week 4: Web Development (Flask/FastAPI)

Goal: Build your first web app with backend logic.

✅ Topics

- HTTP Basics (GET, POST, PUT, DELETE)

- Flask Basics:
    - Routing, Templates (Jinja2)
    - Handling Forms, Sessions
    - Basic REST APIs
- Database Integration (SQLite + SQLAlchemy ORM)

🎯 Practice

- Build a **Flask Blog Website** (CRUD posts)
- Add user registration & login (session-based)

📚 Resources

- 📹 Flask Full Course – freeCodeCamp
- 📘 Flask Docs
- 📘 SQLAlchemy Docs

📅 Week 5: Software Development (GUI Apps)

Goal: Learn to build desktop applications with GUI.

✅ Topics

- Tkinter basics:
    - Window, Labels, Buttons, Entry fields
    - Layout Managers (grid, pack)
    - Events & Callbacks

- File Dialogs, Message Boxes
- Packaging Apps with PyInstaller

🎯 Practice

- Build a **GUI Contact Book App** (using Tkinter + SQLite)
- Convert your Python script to an executable (.exe)

📚 Resources

- 📹 Tkinter Full Course – freeCodeCamp
- 📘 Tkinter Docs
- 📘 PyInstaller Docs

📅 Week 6: Advanced Web Development

Goal: Level up your backend development skills.

✅ Topics

- REST API best practices
- Authentication with JWT (Flask-JWT-Extended)
- Using FastAPI (Async APIs)
- Consuming external APIs in your app
- Deploying web apps (Render, Railway, or Heroku)

🎯 Practice

- Build a **REST API for your Contact Book**

- Deploy your Flask/FastAPI app online

📚 Resources

- 📹 FastAPI Full Course – freeCodeCamp
- 📘 Flask-JWT-Extended Docs
- 📘 Render Deployment Guide

📅 Week 7: Supporting Tools & Best Practices

Goal: Make projects production-ready.

✅ Topics

- Databases: PostgreSQL, MongoDB basics
- Git & GitHub (branches, commits, pull requests)
- Environment Variables (`python-dotenv`)
- Logging & Debugging (pdb, breakpoints)
- Basic Docker (optional but recommended)

🎯 Practice

- Use GitHub for version control on your project
- Connect your app to PostgreSQL instead of SQLite
- Add `.env` for configuration management

📚 Resources

- 📹 Git & GitHub Crash Course – Traversy Media

- 📘 PostgreSQL Docs
- 📹 Docker + Python Crash Course

📅 Week 8: Advanced Concepts + Final Project

Goal: Put everything together into a full project.

✅ Topics

- Asynchronous Programming (`asyncio`, `aiohttp`)
- Design Patterns (MVC, Singleton)
- Security (password hashing, SQL injection prevention)
- CI/CD basics (GitHub Actions)

🎯 Final Project

Pick **ONE full project** and build end-to-end:

- **Option 1:** Web App (Flask/FastAPI + React/Vanilla JS frontend)
    - Example: Blog + Authentication + API + Deployment
- **Option 2:** Desktop App
    - Example: Inventory Manager (PyQt/Tkinter + SQLite + EXE)
- Add:
    - Proper logging
    - Unit tests
    - Documentation (README.md)

📚 Resources

- 📘 Asyncio Docs
- 📘 Refactoring.Guru – Design Patterns in Python
- 📹 GitHub Actions CI/CD Crash Course

🎯 By End of Week 8, You Will Have:

✅ **1–2 complete projects** (web + GUI software)
✅ Experience with **Flask/FastAPI**, **Tkinter**, **SQLAlchemy**, **APIs**, **Git**, **Testing**, **Deployment**
✅ Knowledge of **best practices, security, and environment management**
✅ A **GitHub portfolio** to showcase

Resources

🧭 Topic-Wise Free Resources (Beginner → Advanced)

📅 Week 1 — Python Basics

Goal: Learn syntax, data types, control structures, and functions.

Topics:

- Variables, Data Types, Operators
- Control Flow (if/else, loops)
- Data Structures (list, tuple, dict, set)
- Functions
- File handling (`open`, `read`, `write`)

Resources:

- 📹 Python Full Course for Beginners – freeCodeCamp (4 hrs)
- 📘 Python Docs – Official Tutorial
- 📘 W3Schools Python Basics

📅 Week 2 — Core Python

Goal: Write clean, reusable, and modular code.

Topics:

- OOP (classes, objects, inheritance, polymorphism)

- Modules and Packages
- Exception Handling (custom exceptions)
- Iterators and Generators
- List/Dict comprehensions
- Decorators and Context Managers
- Type hints

Resources:

- 📹 Intermediate Python Programming – freeCodeCamp
- 📘 Python OOP Docs
- 📘 Python Packaging Guide
- 📘 Real Python: Python Decorators

📅 Week 3 — Python Ecosystem & Testing

Goal: Learn testing, environments, and handling external data.

Topics:

- Virtual environments (`venv`)
- pip & `requirements.txt`
- Testing (`unittest`, `pytest`)
- Logging (`logging`)
- Reading/writing JSON, CSV, Excel

- APIs using `requests`

Resources:

- 📘 Pytest Docs
- 📘 Python Logging Docs
- 📘 Requests Docs
- 📹 Python API Tutorial – freeCodeCamp
- 📘 Working with CSV – Python Docs

📅 Week 4 — Web Development (Flask)

Goal: Build your first backend web app.

Topics:

- HTTP basics
- Flask framework
- Templates (Jinja2)
- Routing
- Sessions, Forms
- SQLAlchemy ORM (SQLite/PostgreSQL)

Resources:

- 📹 Flask Full Course – freeCodeCamp
- 📘 Flask Docs
- 📘 SQLAlchemy Docs

- 📘 Jinja2 Template Docs

📅 Week 5 — GUI Software Development

Goal: Create desktop applications.

Topics:

- Tkinter GUI
- Layouts, events, widgets
- File dialogs, message boxes
- Packaging Python apps into .exe (PyInstaller)

Resources:

- 📹 Tkinter Full Course – freeCodeCamp
- 📘 Tkinter Docs
- 📘 PyInstaller Docs
- 📹 Make Python EXE – Tech With Tim

📅 Week 6 — Advanced Web Development (FastAPI)

Goal: Build scalable APIs with modern tools.

Topics:

- REST API best practices
- FastAPI (async APIs)

- Authentication (JWT)
- Connecting external APIs
- Deployment (Render/Railway)

Resources:

- 📹 FastAPI Full Course – freeCodeCamp
- 📘 FastAPI Docs
- 📘 Flask-JWT-Extended Docs
- 📘 Deploy Flask on Render

📅 Week 7 — Supporting Tools & Practices

Goal: Work like a professional developer.

Topics:

- Databases (PostgreSQL, MongoDB)
- Git & GitHub (version control)
- Environment Variables (`python-dotenv`)
- Debugging (pdb)
- Docker basics

Resources:

- 📹 Git & GitHub Crash Course – Traversy Media
- 📘 PostgreSQL Docs
- 📘 python-dotenv Docs

- 📹 Docker + Python – freeCodeCamp

📅 Week 8 — Advanced Concepts & Final Project

Goal: Put everything together.

Topics:

- Async Programming (`asyncio`, `aiohttp`)
- Design Patterns (MVC, Singleton)
- Security (SQL Injection prevention, password hashing with `bcrypt`)
- CI/CD basics (GitHub Actions)
- Documentation (`README.md`, `Sphinx`)

Resources:

- 📘 Asyncio Docs
- 📘 Design Patterns in Python (Refactoring.Guru)
- 📘 bcrypt Docs
- 📹 GitHub Actions CI/CD Crash Course
- 📘 Sphinx Docs

⚡ Bonus: Commonly Used Python Libraries

Purpose	Library	Resource
HTTP Requests	`requests`	Docs

Web Scraping	`BeautifulSoup`, `Selenium`	BeautifulSoup Docs
Databases	`SQLAlchemy`, `psycopg2`, `pymongo`	SQLAlchemy Docs
GUI	`Tkinter`, `PyQt`, `Kivy`	PyQt Tutorial
Testing	`pytest`, `unittest`	pytest Docs
Task Scheduling	`schedule`, `apscheduler`	APScheduler Docs
File Handling	`csv`, `openpyxl`, `pdfplumber`	openpyxl Docs